DECK 0 OF 12 · FRC JAVA CRASH COURSE

# Template gallery

Every slide layout the decks are built from

WPILib 2026 · Java · Command-based

PART 1

# A section divider

Opens a part of the deck

# One idea, in big letters, once or twice a deck.

The accent slide, for one key sentence.

LAYOUT: LEAD

# A short paragraph with supporting points

A lead slide holds one idea in a sentence or two, for when a list would chop the reasoning up.

> A supporting point sits underneath it.

> Inline code turns blue, and **bold** works too.

LAYOUT: POINTS

# A short list when the items really are parallel

- **Subsystems** own hardware.
- **Commands** say what to do with it.
- **Triggers** decide when.

LAYOUT: CARDS

# Two to six cards

### Subsystem

One class per mechanism. It owns the motors.

### Command

One action, with a start, a loop and an end.

### Trigger

Connects a button or a condition to a command.

LAYOUT: STEPS

# Numbered steps, with an optional QR code

1. **Download the installer**
   Grab the 2026 WPILib release for Windows.

2. **Run it**
   Pick “Everything” so VS Code comes along.

3. **Check it**
   Open WPILib VS Code from the desktop shortcut.



**WPILib install guide**

docs.wpilib.org: WPILib Installation

LAYOUT: FIGURE

# A docs screenshot with a numbered key

Numbers in the key match the numbers in the image.




Screenshot: WPILib documentation, CC BY 4.0.

1. **Project type:** Template
2. **Language:** Java
3. **Project base:** Command Robot

PART 2

# Code layouts

LAYOUT: CODE

# Full-width code

RobotContainer.java

```
private void configureBindings() {
  // Sticks read negative when pushed away, and +rotation is counter-clockwise.
  drive.setDefaultCommand(
      drive.driveArcade(
          () -> -MathUtil.applyDeadband(driver.getLeftY(), DEADBAND) * DRIVE_SCALE,
          () -> -MathUtil.applyDeadband(driver.getRightX(), DEADBAND) * TURN_SCALE));
}
```

Pulled by region from a real file in examples/.

LAYOUT: CODENOTES

# Code with numbered callouts

CANDriveSubsystem.java

```
1 // CIMs are BRUSHED motors: the classic KitBot bug.
  private final SparkMax leftLeader =
      new SparkMax(LEFT_LEADER_ID, MotorType.kBrushed);
  private final SparkMax leftFollower =
      new SparkMax(LEFT_FOLLOWER_ID, MotorType.kBrushed);
  private final SparkMax rightLeader =
      new SparkMax(RIGHT_LEADER_ID, MotorType.kBrushed);
  private final SparkMax rightFollower =
      new SparkMax(RIGHT_FOLLOWER_ID, MotorType.kBrushed);

  private final DifferentialDrive drive =
2     new DifferentialDrive(leftLeader, rightLeader);
```

1. **Brushed**
   CIMs are brushed motors.

2. **Two leaders**
   Followers copy their leader.

LAYOUT: COMPARE

# Side by side

**Not this**

```
while (!atGoal()) {
  motor.set(0.5);
}
```

**This**

```
run(() -> motor.set(0.5))
    .until(this::atGoal);
```

LAYOUT: TABLE

# A table with a caption

| Motor | Type | SPARK MAX setting |
|---|---|---|
| CIM | Brushed | kBrushed |
| NEO | Brushless | kBrushless |

Rows stripe automatically.

PART 3

# Recurring layouts

COMMON MISTAKE

# The robot spins in circles

| | |
|---|---|
| **What you see** | Push forward and the robot turns on the spot. |
| **Why it happens** | Both sides run the same direction, but the motors face opposite ways. |
| **The fix** | Invert one side in its config: .inverted(true). |

BEST PRACTICE

# Keep numbers in Constants

> **Every CAN ID, limit and gain gets a name in Constants.java.**

When a motor is swapped at competition you change one line, and the code explains itself.

EXERCISE

# Make it drive

1. Add driveArcade to your subsystem.

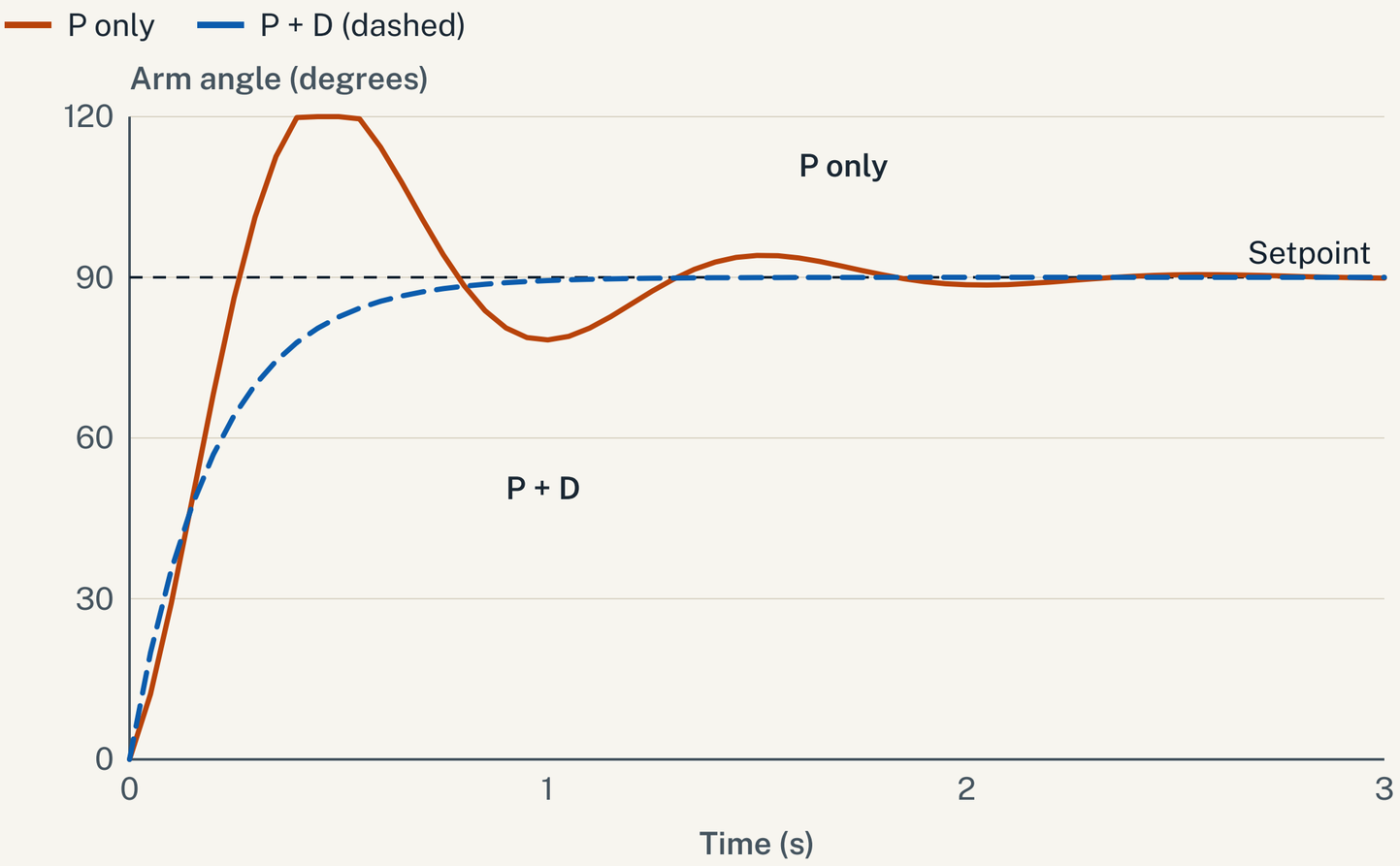
2. Set it as the default command.

3. Deploy with the wheels off the ground.

**Done when: Push the stick forward and both sides spin forward.**

LAYOUT: PLOT

# A chart with callouts

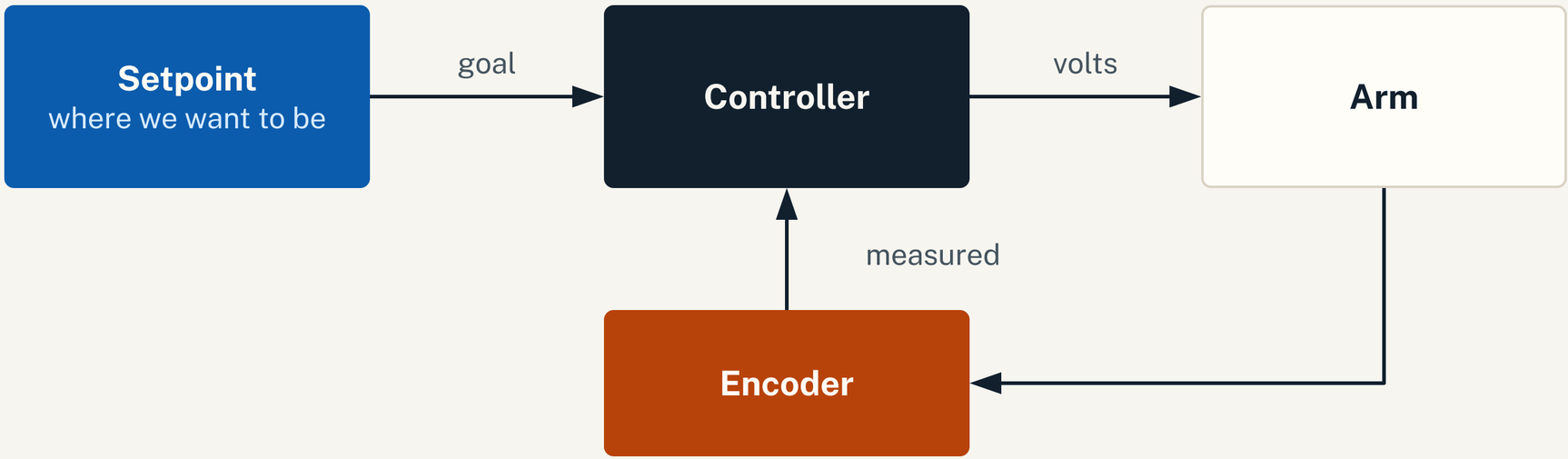



**1 Overshoot**
Too much P swings past the target.

**2 Damping**
D brakes as the error shrinks.

LAYOUT: DIAGRAM

# Blocks and arrows



NEXT SEASON

# What changes for 2027

| In 2026 | Changing for 2027 |
| --- | --- |
| Shuffleboard | Removed: use Elastic |
| edu.wpi.first packages | org.wpilib packages |

Everything here is still true for the 2026 roboRIO.

RECAP

# What this deck covered

- Making a slide from a layout.
- Letting the build catch overflow.

**Next: The real decks.**

REFERENCE

# Official docs



**Zero to Robot**

docs.wpilib.org



**Command-based**

docs.wpilib.org



**AdvantageScope**

docs.advantagescope.org